COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	2/10/2026	**DATE SUBMITTED:**	2/10/2026
NAME:	Fernandez, Cedric S.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Hands-on Activity 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A

CODE

```
#include <iostream>
// stores adjacency list items
  struct adjNode {
    int val, cost;
    adjNode* next;
    };
// structure to store edges
  struct graphEdge {
    int start_ver, end_ver, weight;
    };

  class DiaGraph{
// insert new nodes into adjacency list from given graph
adjNode* getAdjListNode(int value, int weight, adjNode* head) {
  adjNode* newNode = new adjNode;
  newNode->val = value;
  newNode->cost = weight;
  newNode->next = head; // point new node to current head

  return newNode;
  }

  int N; // number of nodes in the graph
  public:
  adjNode **head; //adjacency list as array of pointers
// Constructor
DiaGraph(graphEdge edges[], int n, int N) {
// allocate new node
head = new adjNode*[N]();
this->N = N;
// initialize head pointer for all vertices
for (int i = 0; i < N; ++i)
  head[i] = nullptr;
// construct directed graph by adding edges to it
  for (unsigned i = 0; i < n; i++) {
    int start_ver = edges[i].start_ver;
    int end_ver = edges[i].end_ver;
    int weight = edges[i].weight;
// insert in the beginning
adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
// point head pointer to new node
head[start_ver] = newNode;
    }
  }
```

OUTPUT

```
Graph adjacency list
(start_vertex, end_vertex,weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

```
// Destructor
  ~DiaGraph() {
    for (int i = 0; i < N; i++)
    delete[] head[i];
    delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i){
  while (ptr != nullptr) {
    std::cout << "(" << i << ", " << ptr->val
    << ", " << ptr->cost << ") ";
    ptr = ptr->next;
    }
    std::cout << std::endl;
    }
// graph implementation
int main() {
// graph edges array.
graphEdge edges[] = {
// (x, y, w) -> edge from x to y with weight w
{0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
};
int N = 6; // Number of vertices in the graph
// calculate number of edges
int n = sizeof(edges)/sizeof(edges[0]);
// construct graph
DiaGraph diagraph(edges, n, N);
// print adjacency list representation of graph
std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex,weight):"<<std::endl;
  for (int i = 0; i < N; i++) {
// display adjacent vertices of vertex i
    display_AdjList(diagraph.head[i], i);
  }
return 0;
}
```

ILOB

CODE	OUTPUT

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>
template <typename T>
class Graph;

template <typename T>
struct Edge
{
  size_t src;
  size_t dest;
  T weight;
// To compare edges, only compare their weights,
// and not the source/destination vertices
  inline bool operator<(const Edge<T> &e) const {
    return this->weight < e.weight;
  }
  inline bool operator>(const Edge<T> &e) const {
    return this->weight > e.weight;
  }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{

for (auto i = 1; i <= G.vertices(); i++) {
  os << i << ":\t";
  auto edges = G.outgoing_edges(i);
    for (auto &e : edges)
    os << "{" << e.dest << ": " << e.weight << "}, ";
    os << std::endl;
    }
  return os;
}

//THIS HERE
template <typename T>
class Graph {
public:
// Initialize the graph with N vertices
  Graph(size_t N) : V(N) {}

// Return number of vertices in the graph
```

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

```
auto vertices() const {
  return V;
  }

// Return all edges in the graph
auto &edges() const {
  return edge_list;
  }

void add_edge(const Edge<T> &e) {
// Check if the source and destination vertices are within range
  if (e.src >= 1 && e.src <= V &&
    e.dest >= 1 && e.dest <= V)
    edge_list.emplace_back(e);
  else
    std::cerr << "Vertex out of bounds" << std::endl;
}
// Returns all outgoing edges from vertex v
auto outgoing_edges(size_t v) const {
  std::vector<Edge<T>> edges_from_v;
  for (auto &e : edge_list) {
    if (e.src == v)
      edges_from_v.emplace_back(e);
      }
      return edges_from_v;
}
// Overloads the << operator so a graph be written directly to a stream
// Can be used as std::cout << obj << std::endl;
template <typename U>
friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
private:
  size_t V; // Stores number of vertices in graph
  std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) {
  std::stack<size_t> stack;
  std::vector<size_t> visit_order;
  std::set<size_t> visited;
  stack.push(1); // Assume that DFS always starts from vertex ID 1
  while (!stack.empty())
  {
    auto current_vertex = stack.top();
    stack.pop();
```

```
    // If the current vertex hasn't been visited in the past
      if (visited.find(current_vertex) == visited.end()) {
        visited.insert(current_vertex);
        visit_order.push_back(current_vertex);
        for (auto e : G.outgoing_edges(current_vertex))
        {
    // If the vertex hasn't been visited, insert it in the stack.
          if(visited.find(e.dest) == visited.end()) {
            stack.push(e.dest);
            }
          }
      }
    }
  return visit_order;
}

template <typename T>
auto create_reference_graph() {
  Graph<T> G(8);
  std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
  edges[1] = {{2, 0}, {5, 0}};
  edges[2] = {{1, 0}, {5, 0}, {4, 0}};
  edges[3] = {{4, 0}, {7, 0}};
  edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
  edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
  edges[6] = {{4, 0}, {7, 0}, {8, 0}};
  edges[7] = {{3, 0}, {6, 0}};
  edges[8] = {{4, 0}, {5, 0}, {6, 0}};
  for (auto &i : edges)
    for (auto &j : i.second)
      G.add_edge(Edge<T>{i.first, j.first, j.second});
  return G;
}

template <typename T>
void test_DFS() {
// Create an instance of and print the graph
  auto G = create_reference_graph<unsigned>();
  std::cout << G << std::endl;
// Run DFS starting from vertex ID 1 and print the order
// in which vertices are visited.
  std::cout << "DFS Order of vertices: " << std::endl;
  auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
      std::cout << v << std::endl;
```

```
}
int main() {
  using T = unsigned;
  test_DFS<T>();

  return 0;
}
```

ILOB.2

CODE	OUTPUT

```
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge {
  size_t src;
  size_t dest;
  T weight;
  inline bool operator<(const Edge<T> &e) const {
  return this->weight < e.weight;
}
  inline bool operator>(const Edge<T> &e) const {
  return this->weight > e.weight;
}
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G) {
  for (auto i = 1; i < G.vertices(); i++) {
  os << i << ":\t";
  auto edges = G.outgoing_edges(i);
    for (auto &e : edges)
    os << "{" << e.dest << ": " << e.weight << "}, ";
    os << std::endl;
  }
return os;
}

template <typename T>
class Graph {

  public:
  Graph(size_t N) : V(N) {}
  auto vertices() const {
  return V;
}

auto &edges() const {
  return edge_list;
}

void add_edge(const Edge<T> &e) {

  if (e.src >= 1 && e.src <= V &&
    e.dest >= 1 && e.dest <= V)
    edge_list.emplace_back(e);
    else
    std::cerr << "Vertex out of bounds" << std::endl;
  }

  auto outgoing_edges(size_t v) const {
```

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

```
    std::vector<Edge<T>> edges_from_v;
    for (auto &e : edge_list) {
      if (e.src == v)
      {
edges_from_v.emplace_back(e);
    }
}
return edges_from_v;
}

template <typename U>
friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
  private:
  size_t V;
  std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph() {
  Graph<T> G(9);

  std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

  edges[1] = {{2, 2}, {5, 3}};
  edges[2] = {{1, 2}, {5, 5}, {4, 1}};
  edges[3] = {{4, 2}, {7, 3}};
  edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
  edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
  edges[6] = {{4, 4}, {7, 4}, {8, 1}};
  edges[7] = {{3, 3}, {6, 4}};
  edges[8] = {{4, 5}, {5, 3}, {6, 1}};

  for (auto &i : edges)
    for (auto &j : i.second)
    G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
  }

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
  std::queue<size_t> queue;
  std::vector<size_t> visit_order;
  std::set<size_t> visited;
  queue.push(1); // Assume that BFS always starts from vertex ID 1
  while (!queue.empty()) {
    auto current_vertex = queue.front();
    queue.pop();
// If the current vertex hasn't been visited in the past
if (visited.find(current_vertex) == visited.end()) {
  visited.insert(current_vertex);
  visit_order.push_back(current_vertex);
    for (auto e : G.outgoing_edges(current_vertex))
      if (visited.find(e.dest) == visited.end())
  queue.push(e.dest);
  }
```

```
}
return visit_order;
}

template <typename T>
void test_BFS() {
// Create an instance of and print the graph
auto G = create_reference_graph<unsigned>();
std::cout << G << std::endl;
// Run BFS starting from vertex ID 1 and print the order
// in which vertices are visited.
std::cout << "BFS Order of vertices: " << std::endl;
auto bfs_visit_order = breadth_first_search(G, 1);
for (auto v : bfs_visit_order)
  std::cout << v << std::endl;
}

int main() {
  using T = unsigned;
  test_BFS<T>();

  return 0;
}
```

OBSERVATIONS:

The following code for graph traversal DFS and BFS showed and demonstrated how stacks and queues change the order on which each nodes are visited in a graph but before it was executed I had to trouble shoot the code provide mainly for the template types for printing the function and also made sure the loops were correctly bounded so that it covers the nodes needed properly in the graph that was done.

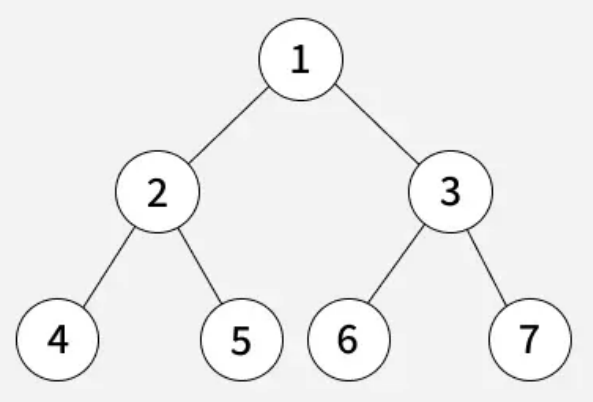
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

Depth-First Search (DFS) would be the best for this situation because it works by basically going as deep as it can in a single path until it can't anymore then after it backtracks to the most recent vertex to explore unvisited places..

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.



Preorder: 1 2 4 5 3 6 7
Postorder:4 5 2 6 7 3 1
Inorder: 4 2 5 1 6 3 7

For the following the most equivalent for DFS is the Preorder traversal because just like DFS it goes deep down until it can't anymore and goes to backtrack, which once more is similar to preorder cause it goes down up to 4 or the leaf node then after it moves to the right leaf child.

3. In the performed code, what data structure is used to implement the Breadth First Search?

From the previous code in the ILO regarding the BFS the data structure used to implement it was queue as shown below to ensure that the vertices that were done were in the exact order they've been visited giving us the ability to explore the graph level by level before going down

```cpp
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) {
  std::queue<size_t> queue;
  std::vector<size_t> visit_order;
  std::set<size_t> visited;
  queue.push(1); // Assume that BFS always starts from vertex ID 1
  while (!queue.empty()) {
    auto current_vertex = queue.front();
    queue.pop();
// If the current vertex hasn't been visited in the past
if (visited.find(current_vertex) == visited.end()) {
  visited.insert(current_vertex);
  visit_order.push_back(current_vertex);
    for (auto e : G.outgoing_edges(current_vertex))
      if (visited.find(e.dest) == visited.end())
  queue.push(e.dest);
  }
}
return visit_order;
}
```

4. How many times can a node be visited in the BFS?

When it comes to BFS a node can only be visited one time because after that it would be marked as visited and removed from the queue.

CONCLUSION

This activity gave me a deeper insight into how graphs work when it comes to c++ and how to implements their traversals like how the use of stacks can give us a DFS because of its backtracking and queue giving us the BFS cause of how it does every level by FIFO. In addition to this I also learned how to prevent things like accidently making infinite loops when troubleshooting and debugging template functions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, Explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explanation of concept, Trouble shooting explanations**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.